Bachelor of Technology (Computer Science & Engineering)

Final Semester Dissertation / Major Project Report

CampusIQ: The AI Student Operating System

Submitted in partial fulfillment of the requirements for the award of the degree of

Bachelor of Technology (Computer Science & Engineering)

Submitted by

Danish Rizwan
Enrollment Number: _____

Under the supervision of

Supervisor Name: _____

Department of Computer Science & Engineering
School of Engineering Sciences & Technology
Jamia Hamdard, New Delhi

Academic Year: 2025-2026

Certificate

This is to certify that the Dissertation / Major Project Report entitled **CampusIQ: The AI Student Operating System** is a bonafide record of the project work carried out by **Danish Rizwan**, Enrollment Number _____, in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology (Computer Science & Engineering) at the Department of Computer Science & Engineering, School of Engineering Sciences & Technology, Jamia Hamdard, New Delhi.

The work reported in this dissertation has been carried out under my supervision and guidance. To the best of my knowledge, the work is original and has not been submitted to any other university or institution for the award of any degree, diploma, associateship, fellowship, or similar title.

Signature of Supervisor	**Department Office Seal**
Name: _____ Date: _____	Department of CSE Jamia Hamdard, New Delhi

Declaration

I, **Danish Rizwan**, a student of **Bachelor of Technology (Computer Science & Engineering)**, Enrollment Number _____, hereby declare that the dissertation entitled **CampusIQ: The AI Student Operating System**, which is being submitted by me to the Department of Computer Science & Engineering, School of Engineering Sciences & Technology, Jamia Hamdard, New Delhi, in partial fulfillment of the requirement for the award of the degree of Bachelor of Technology (Computer Science & Engineering), is my original work.

I further declare that this work has not been submitted anywhere else for the award of any Degree, Diploma, Associateship, Fellowship, or other similar title or recognition. All references, tools, APIs, frameworks, and supporting materials used in the project have been acknowledged appropriately in the bibliography.

Signature and Name of Applicant	**Date and Place**
Danish Rizwan	Date: 26 April 2026 Place: New Delhi

Acknowledgements

I express my sincere gratitude to my project supervisor for continuous guidance, valuable feedback, and encouragement throughout the planning, design, development, testing, and documentation of this project. Their advice helped shape CampusIQ from an initial idea into a complete full-stack academic software system.

I am thankful to the Department of Computer Science & Engineering, School of Engineering Sciences & Technology, Jamia Hamdard, New Delhi, for providing the academic environment and technical foundation required to complete this work. I also acknowledge the documentation and developer communities around Next.js, React, Prisma, Clerk, PostgreSQL, and Google Gemini, whose resources were helpful during implementation.

Finally, I thank my family, friends, and peers for their support, patience, and motivation during the development of this final year project.

Signature	**Date and Place**
Danish Rizwan	Date: 26 April 2026 Place: New Delhi

List of Abbreviations

Abbreviation	Full Form
AI	Artificial Intelligence
API	Application Programming Interface
B.Tech	Bachelor of Technology
CSE	Computer Science & Engineering
DBMS	Database Management System
ERD	Entity Relationship Diagram
HTTP	Hypertext Transfer Protocol
IEEE	Institute of Electrical and Electronics Engineers
JSON	JavaScript Object Notation
LLM	Large Language Model
ORM	Object Relational Mapping
PDF	Portable Document Format
SRS	Software Requirements Specification
UI	User Interface
UML	Unified Modeling Language

List of Tables

Table No.	Caption	Page No.
Table 5.1	Functional Requirements	12
Table 5.2	Non-functional Requirements	13
Table 5.3	Use Case Specifications	15
Table 6.1	Cost and Effort Estimation	20
Table 10.1	Input Validations and Checks	27
Table 11.1	Description of Reports	29
Table 12.1	White Box Test Cases	31
Table 12.2	Black Box Test Cases	32

List of Figures

Figure No.	Caption	Page No.
Figure 5.1	UML Use Case Diagram	14
Figure 7.1	UML Class Diagram	22
Figure 8.1	Entity Relationship Diagram	24
Figure 9.1	Mock Exam Grading Sequence Diagram	26
Figure 12.1	Gantt Chart	33
Figure 13.1-13.14	Application screenshots	35 onwards

1. Title

The title of the dissertation is **CampusIQ: The AI Student Operating System**. CampusIQ is a full-stack, AI-assisted academic management platform designed for students who need one unified environment for career planning, semester roadmaps, syllabus intelligence, examination preparation, mock assessment, class scheduling, task management, and performance analytics.

The project has been developed as a modern web application using Next.js 16, React 19, Prisma, PostgreSQL, Clerk authentication, Google Gemini AI, Zustand, Recharts, and a responsive custom CSS interface. It demonstrates how artificial intelligence can be integrated into academic workflows to reduce manual planning effort and provide personalized guidance.

2. Objective

The objective of CampusIQ is to convert a fragmented student workflow into a unified, intelligent, and measurable academic operating system. Students commonly use separate tools for timetables, notes, syllabus tracking, exam preparation, career research, and performance review. This separation creates duplicated effort and weak feedback loops.

The primary objectives are: secure onboarding, adaptive semester roadmap generation, syllabus analysis, exam preparation planning, 75-mark mock exam generation, AI-assisted grading, task and schedule management, analytics, and an AI study assistant.

The scope includes a complete student-facing web platform, protected dashboard routes, API route handlers, database persistence, AI generation endpoints, analytics aggregation, and a responsive user interface. The system complements institutional ERP systems by providing a student-owned planning and preparation layer.

3. Introduction

Education technology has evolved from static information systems to adaptive digital environments. Traditional academic tools often record information but rarely interpret it. A timetable application may store class timings, a note application may store text, and a syllabus PDF may list topics, but these tools usually do not connect the information to career goals, examination readiness, or learning progress.

CampusIQ is built around the idea that a student needs an academic command center. The system connects four major data flows: the student's identity and profile, the curriculum or syllabus, semester-wise planning, and performance evidence from tasks, mocks, and evaluations. Once these data flows exist together, AI can help generate roadmaps, explain topics, recommend preparation plans, and evaluate answers.

The application uses Next.js App Router for routing and server-side API handlers. Clerk provides authentication. Prisma with PostgreSQL stores structured data. Google Gemini is used for natural language reasoning and JSON generation. The interface follows a dashboard model with Roadmap, Career, Syllabus, Assistant, Exam Prep, Mock Exam, Timetable, Analytics, and Settings modules.

4. Problem Statement

Students in technical courses face a combination of academic, organizational, and career-planning challenges. Their learning resources are scattered across PDF syllabi, classroom notes, online tutorials, exam papers, placement advice, spreadsheets, and messaging groups.

The problem is not merely the absence of information. In most cases, students have too much unstructured information. The real challenge is converting that information into a personalized plan and then measuring progress against it.

CampusIQ addresses the lack of an integrated, AI-assisted academic platform that can transform syllabus and performance data into personalized roadmaps, exam simulations, study assistance, and analytics for students in computer science and related programs.

5. SRS in IEEE Format

5.1 Purpose

This Software Requirements Specification describes the functional and non-functional requirements for CampusIQ. It defines the behavior expected from the system, the major actors, use cases, data interactions, and quality constraints.

5.2 Functional Requirements

Req. ID	Requirement	Description	Priority
FR-01	Authentication	Allow students to sign up, sign in, and sign out securely.	High
FR-02	User Sync	Synchronize authenticated Clerk users into the local database.	High
FR-03	Roadmap Generation	Generate an initial semester roadmap using target career, course, duration, skill level, and study hours.	High
FR-04	Roadmap Evaluation	Evaluate semester performance and generate the next semester roadmap.	High
FR-05	Syllabus Analysis	Analyze syllabus text/PDF-derived content and generate structured study topics.	High
FR-06	Mock Exam	Generate, store, display, grade, and retake mock exams.	High
FR-07	AI Assistant	Provide a study assistant that answers academic queries.	Medium
FR-08	Timetable and Tasks	Allow class schedule and task management.	Medium
FR-09	Analytics	Aggregate roadmap and mock exam data into visual reports.	High

5.3 Non-functional Requirements

Req. ID	Category	Requirement
NFR-01	Security	Dashboard and API routes must be protected by authenticated sessions.
NFR-02	Usability	The interface must be responsive and usable on desktop and mobile devices.
NFR-03	Reliability	The system should provide fallback responses for selected AI failures.
NFR-04	Maintainability	Code should follow modular Next.js route and component structure.
NFR-05	Performance	Dashboard pages should fetch only necessary internal API data.
NFR-06	Scalability	Database models should support multiple users and one-to-many academic records.

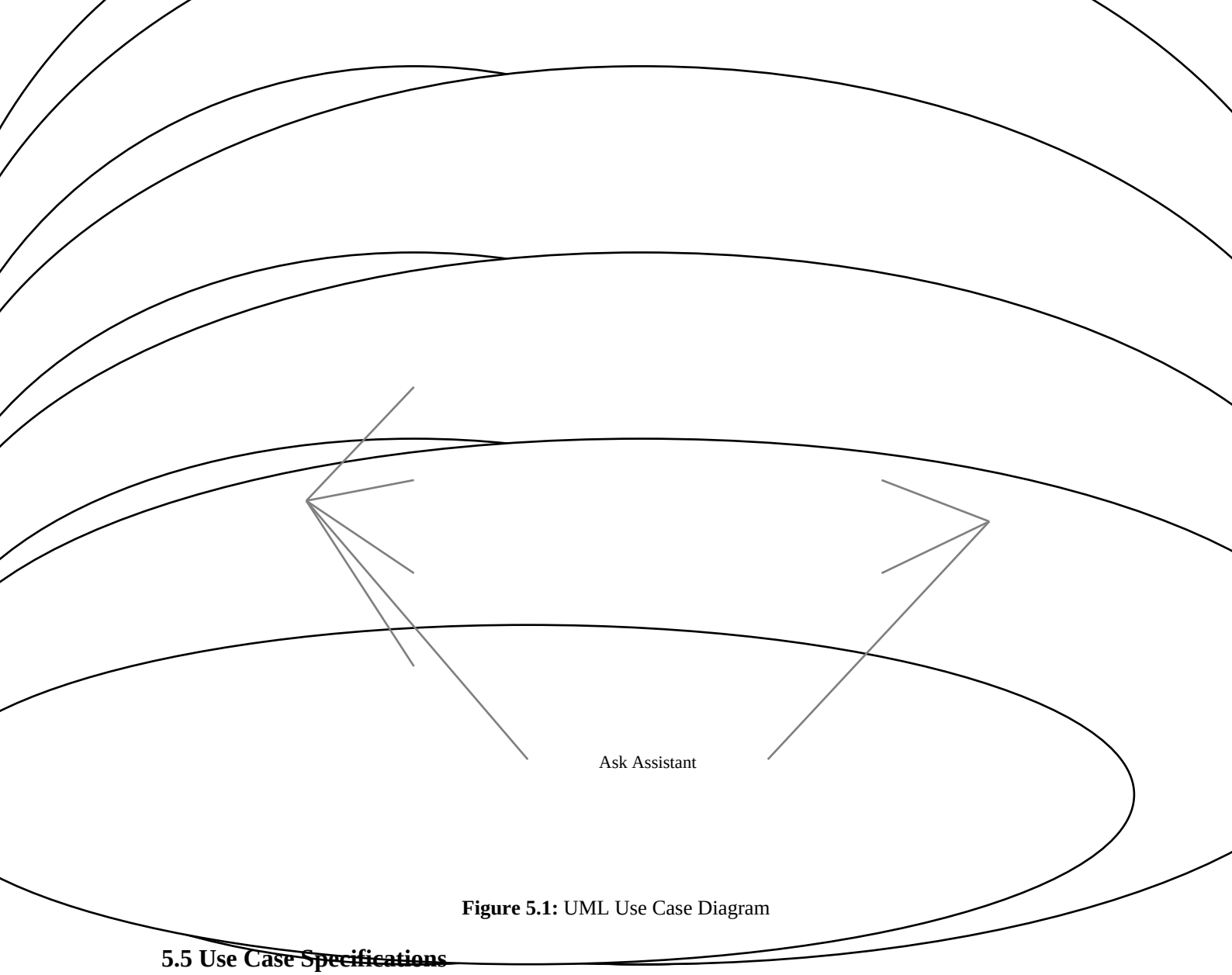

Figure 5.1: UML Use Case Diagram

5.5 Use Case Specifications

Use Case	Actor	Pre-condition	Basic Path	Post-condition
Register and authenticate	Student	Valid email or social login	Sign up/sign in through Clerk and sync user profile.	Dashboard session is available.
Generate roadmap	Student, AI Core	Authenticated user enters course/career data.	Gemini creates subjects, skills, projects, setup, and 24-week plan.	Semester 1 roadmap becomes ACTIVE.

Use Case	Actor	Pre-condition	Basic Path	Post-condition
Evaluate semester	Student, AI Core	Active roadmap exists.	Submit performance metrics; system calculates weighted score and generates next semester.	Current semester COMPLETED and next semester ACTIVE.
Analyze syllabus	Student, AI Core	Text or PDF syllabus is available.	System extracts topics, difficulty, documentation, resources, and questions.	Syllabus topics are saved.
Take mock exam	Student, AI Core	Subject and syllabus context exist.	Generate exam, answer questions, submit for grading.	Score and feedback are stored.
Ask assistant	Student, AI Core	Authenticated user.	User submits topic/question and receives short or detailed answer.	Chat history is updated.

6. Cost/Effort Estimation

Cost and effort estimation is prepared for a student-led final year software project. The development was performed by one full-stack developer using open-source frameworks and free or student-accessible cloud services. Monetary cost is low, while effort cost is measured in analysis, design, implementation, testing, and documentation time.

Activity	**Estimated Duration**	**Effort Description**	**Output**
Requirement analysis	1 week	Study academic workflow, identify modules, prepare feature list.	Problem definition and SRS
UI/UX design	1.5 weeks	Design landing page, dashboard shell, cards, forms, and themes.	Responsive interface
Database design	1 week	Design Prisma models for users, roadmaps, tasks, syllabus, exams, chats, and analytics.	Database schema
Authentication setup	0.5 week	Integrate Clerk, protected proxy, login, signup, user sync.	Secure access flow
AI integration	2 weeks	Prepare Gemini prompts, JSON schemas, fallbacks, roadmap/exam/syllabus/chat logic.	AI service layer
Dashboard modules	3 weeks	Build roadmap, syllabus, assistant, exam, mock exam, timetable, analytics pages.	Functional modules
Testing and debugging	1.5 weeks	Run build checks, validate API behavior, test UI flows, handle errors.	Stable release
Documentation	1 week	Prepare report, diagrams, screenshots, testing tables, bibliography.	Dissertation

7. UML Class Diagram

The UML class model is derived from the Prisma schema and the application modules. The main aggregate root is User. A User can have one CareerProfile and many SemesterRoadmap, Task, ClassSchedule, ChatSession, ExamPrep, SyllabusTopic, and MockExam records. JSON fields are used where AI-generated structures are flexible.

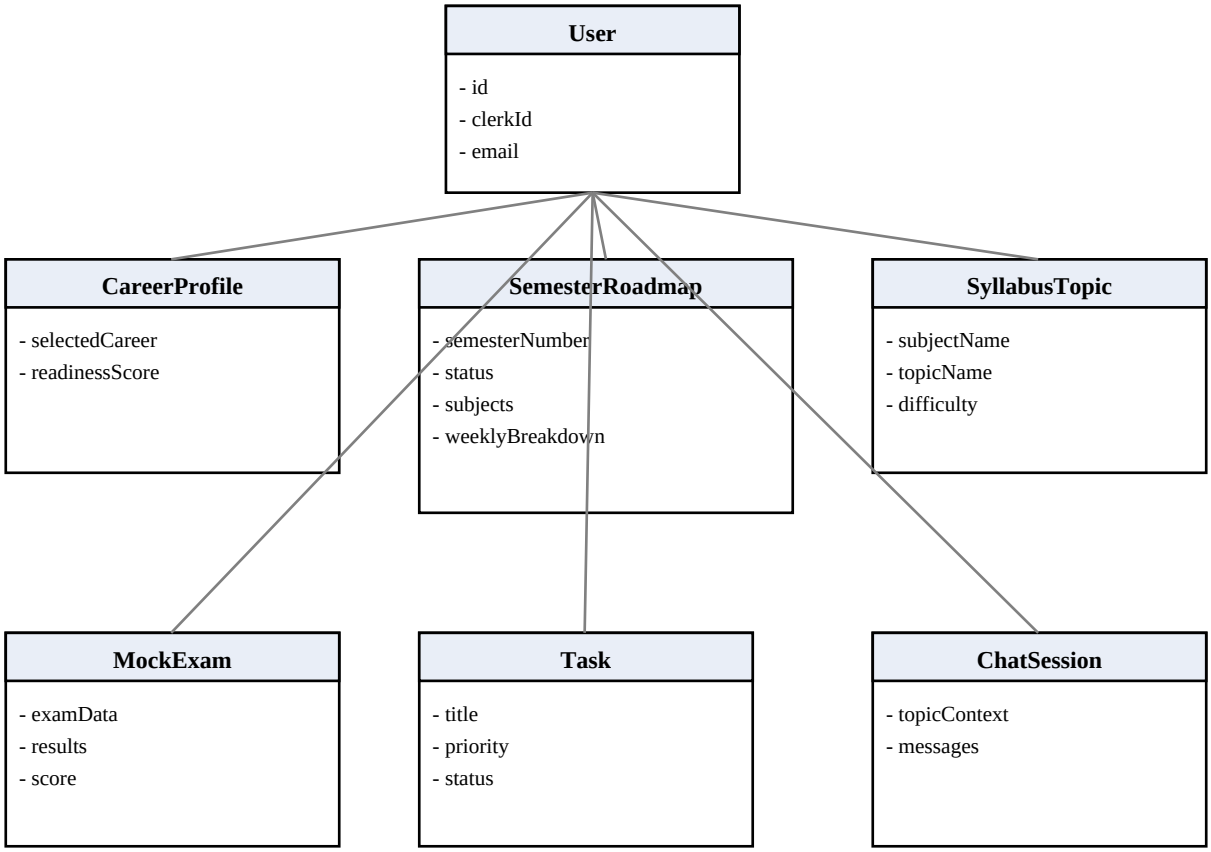

Figure 7.1: UML Class Diagram

8. Entity Relationship Diagram

The database is implemented using PostgreSQL and Prisma. User is the central entity, and most academic records depend on it through foreign key relationships with cascade delete.

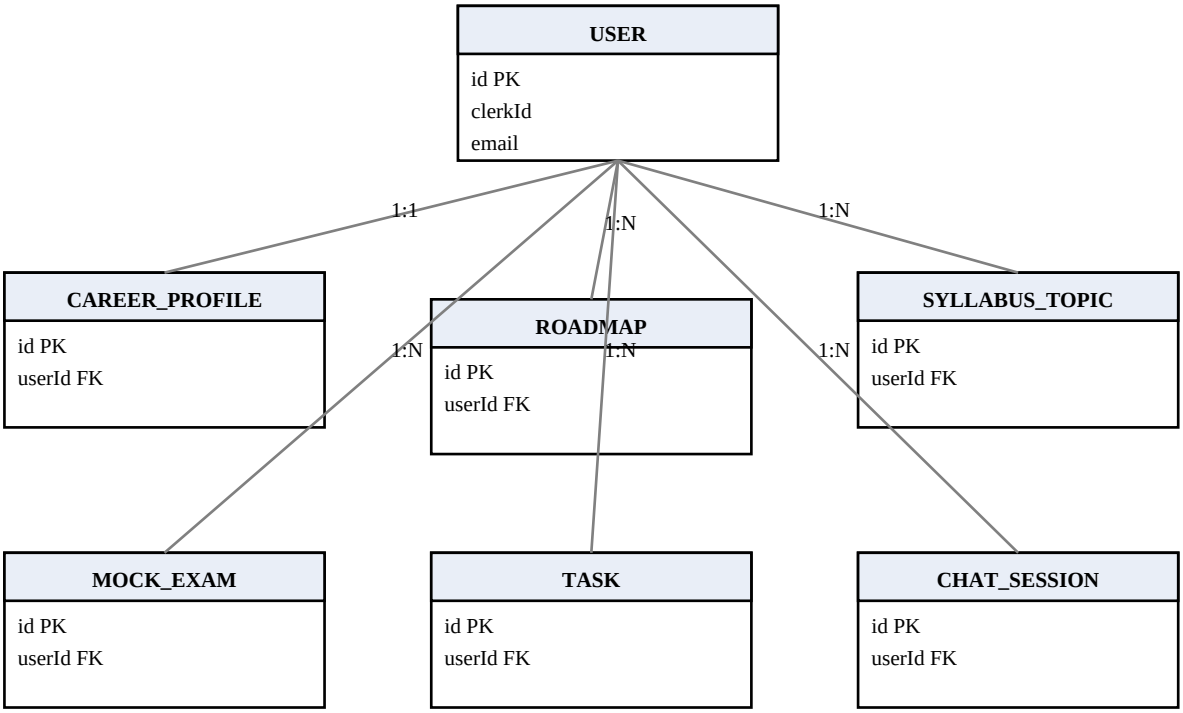

Figure 8.1: Entity Relationship Diagram

• One User has zero or one CareerProfile.

• One User has many SemesterRoadmap records, one for each generated or completed semester.

• One User has many SyllabusTopic records because each syllabus can produce multiple topics.

• One User has many MockExam records to preserve attempts and grading output.

• One User has many Task, ClassSchedule, ChatSession, and ExamPrep records.

9. Activity / Sequence Diagram

The following sequence diagram describes the mock exam grading workflow. This flow combines user interaction, protected API access, database ownership checks, AI evaluation, fallback logic, persistence, and analytics.

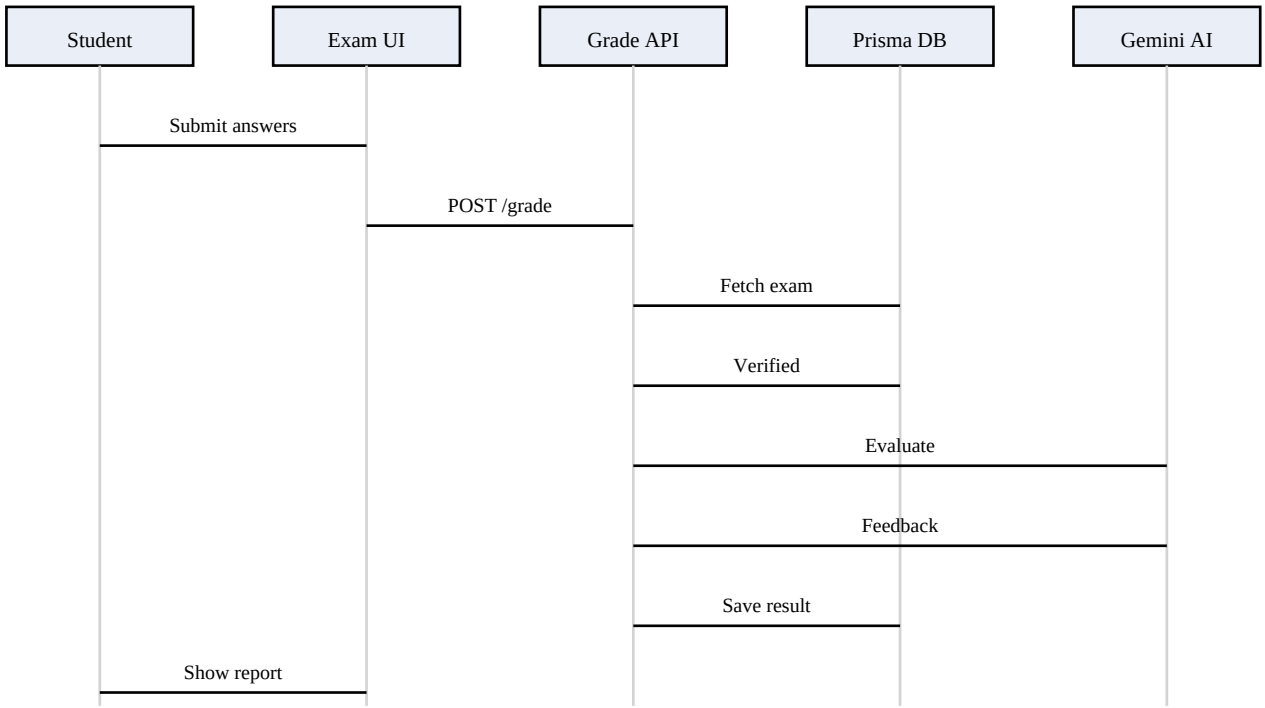

Figure 9.1: Mock Exam Grading Sequence Diagram

10. Descriptions of Different Input Validations and Checks

Input validation is necessary because CampusIQ processes user-generated academic content, AI prompts, and database-changing operations. The most important validation objective is to ensure that all operations are performed only for the authenticated user.

Area	Validation / Check	Implementation	Purpose
Authentication	Clerk session must exist.	src/proxy.ts and route handlers call auth().	Prevents anonymous access.
User sync	Clerk user must have id and email.	/api/user/sync validates clerkId and email.	Avoids orphan records.
Roadmap generation	Course, duration, career, skill level, and hours required.	/api/roadmap/generate rejects incomplete requests.	Ensures useful AI prompts.
Duplicate roadmap	Existing roadmap cannot be overwritten unless overwrite is true.	Transaction checks existing roadmap.	Protects progress.
Syllabus analysis	Text content is required.	/api/syllabus/analyze returns 400 for missing text.	Prevents empty AI calls.
Mock grading	Exam must belong to current user and not already be completed.	/api/exam/mock/[id]/grade checks ownership/status.	Prevents duplicate grading.
Tasks/schedule	Records filtered by current user id.	/api/tasks and /api/schedule query by userId.	Maintains tenant isolation.

11. Description of Different Reports

CampusIQ produces several dashboard reports computed from persisted records and AI-generated structures.

Report	Input Data	Output / Interpretation
Dashboard Overview	User, active roadmap, tasks, latest mock exam, analytics summary	Greeting, semester status, task count, mock readiness, and career progress.
Roadmap Timeline	SemesterRoadmap subjects, skills, projects, weeklyBreakdown, environmentSetup	Semester goals, weekly plan, projects, tools, resources, and recommendations.
Syllabus Topic Report	SyllabusTopic subject, topic, difficulty, documentation, practice questions	Study cards and detailed topic pages.
Mock Exam Result Report	MockExam examData, answers, results, score, totalMarks	Marks, feedback, correctness, and readiness insight.
Analytics Report	Completed roadmaps and graded mock exams	Charts, trends, weakest metric, career readiness, and mock history.
Career Assessment Report	Interests, skills, traits, salary and work-life preferences	Career recommendations and readiness score.

12. Sample Test Cases

12.1 White Box Testing

Test ID	Module	Input / Condition	Expected Internal Path	Result
WB-01	Roadmap fallback	Gemini API unavailable or JSON parse fails	generateInitialRoadmap catches error and returns local 24-week mock roadmap	PASS
WB-02	Performance score	80,70,75,90,60	Weighted score = 76.25	PASS
WB-03	Mock grading fallback	Subjective answer above 50 words	Local evaluator awards approximate 75 percent marks	PASS
WB-04	Analytics	No semesters and no mocks	Endpoint returns hasData=false and setup guidance	PASS

For the roadmap evaluation module, major decision points are authentication check, roadmapId validation, final semester check, and normal next-semester generation. The approximate cyclomatic complexity is decision points + 1, which gives V(G) = 4. Independent paths include unauthorized request, missing roadmapId, final semester completion, and normal next-semester generation.

12.2 Black Box Testing

Test ID	Use Case	Input / Action	Expected Output	Result
BB-01	Login access control	Unauthenticated user opens /dashboard	User is protected by Clerk	PASS
BB-02	Create roadmap	Submit without targetCareer	API returns 400 Missing fields	PASS
BB-03	Theme toggle	Click theme control	App switches light/dark variables	PASS

Test ID	Use Case	Input / Action	Expected Output	Result
BB-04	Generate mock exam	Select subject and generate	Pending mock exam record appears	PASS
BB-05	Delete task	Click delete on existing task	Task disappears	PASS
BB-06	Analyze syllabus	Paste syllabus and submit	Structured topics appear	PASS

12.3 Gantt Chart

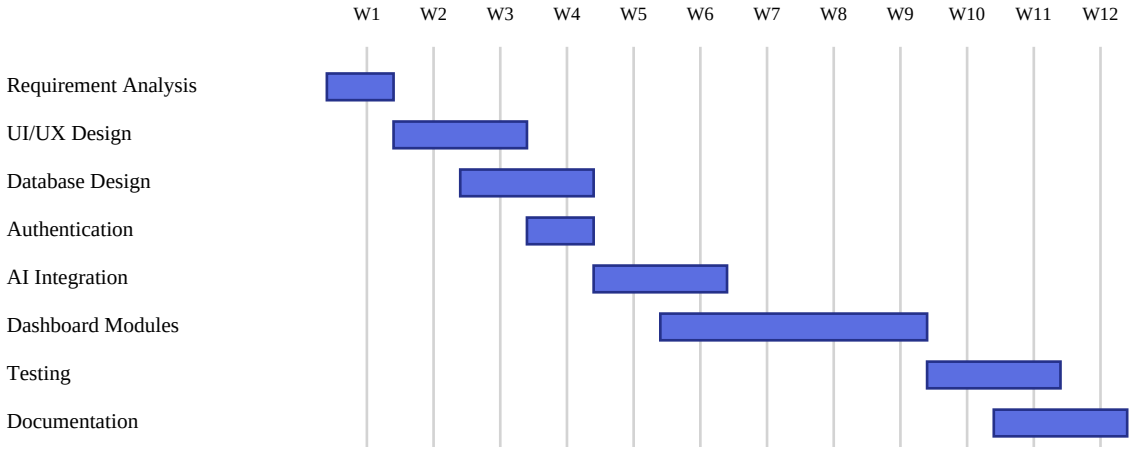

Figure 12.1: Gantt Chart Depicting Project Timeline

13. Snapshots of Different Input and Output Screens

This chapter includes screenshots from the implemented CampusIQ application. The screenshots demonstrate the public landing experience, authentication, dashboard cockpit, roadmap generation, syllabus hub, mock exam engine, and analytics reports.

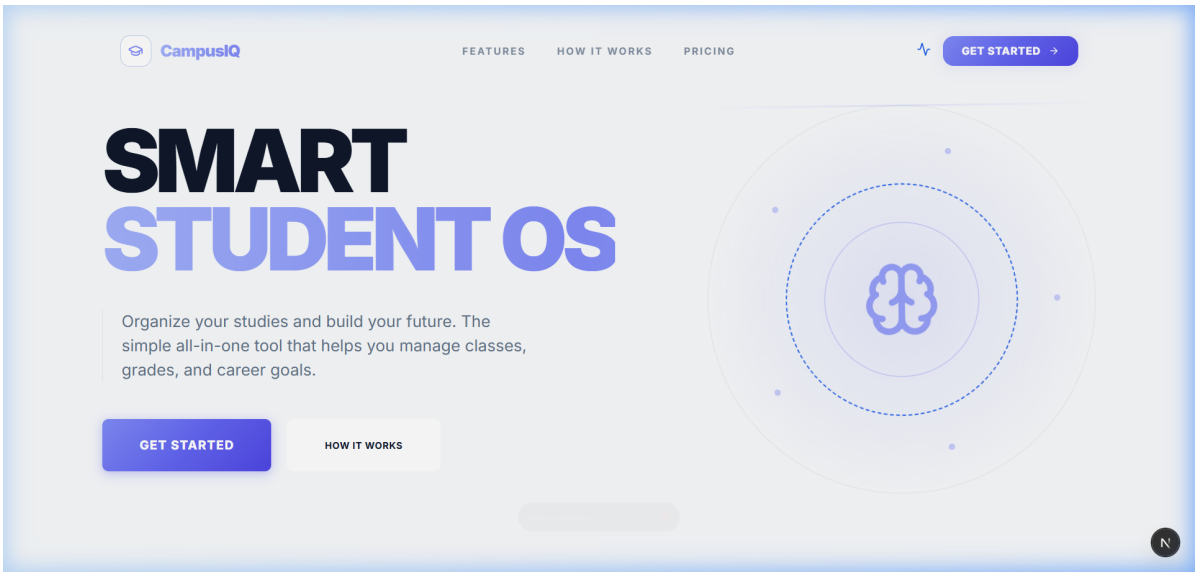

Figure 13.1: CampusIQ landing page in dark theme

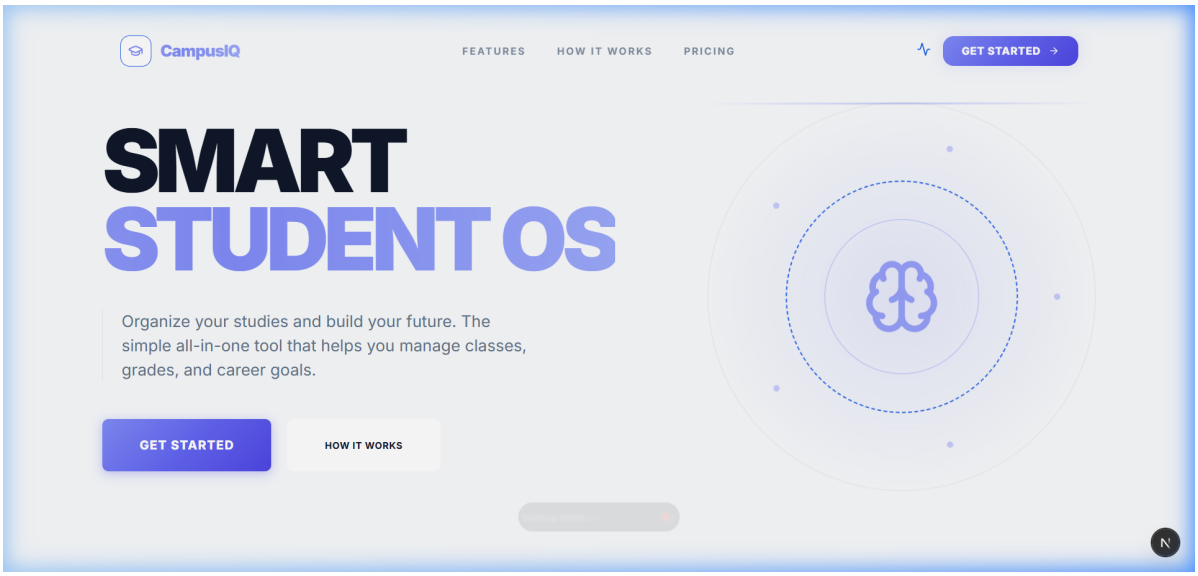

Figure 13.2: CampusIQ landing page in light theme

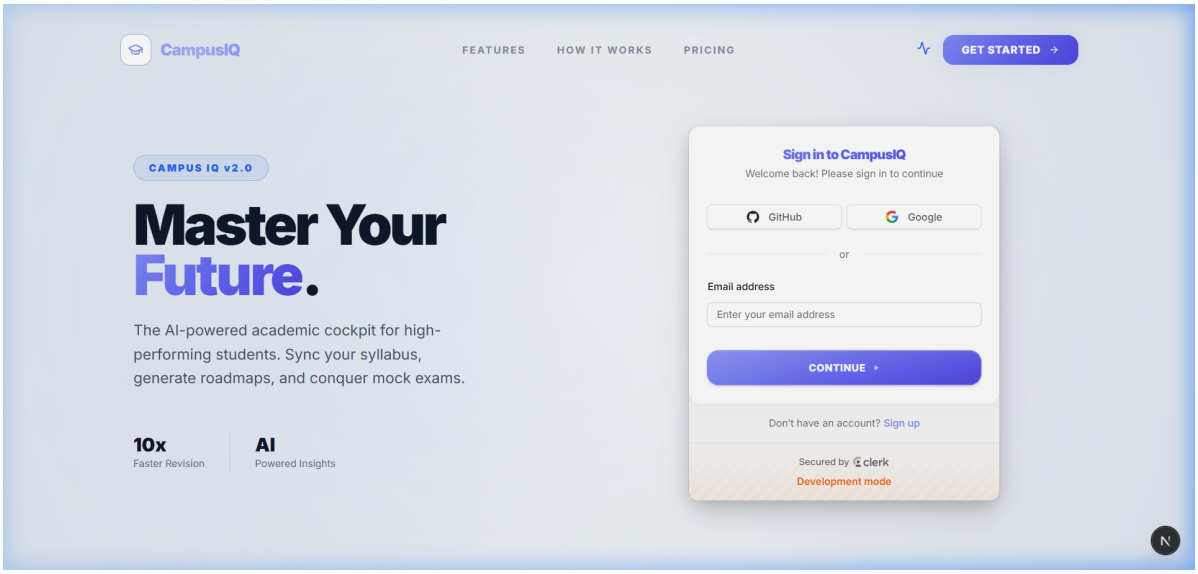

Figure 13.3: Secure sign-in page

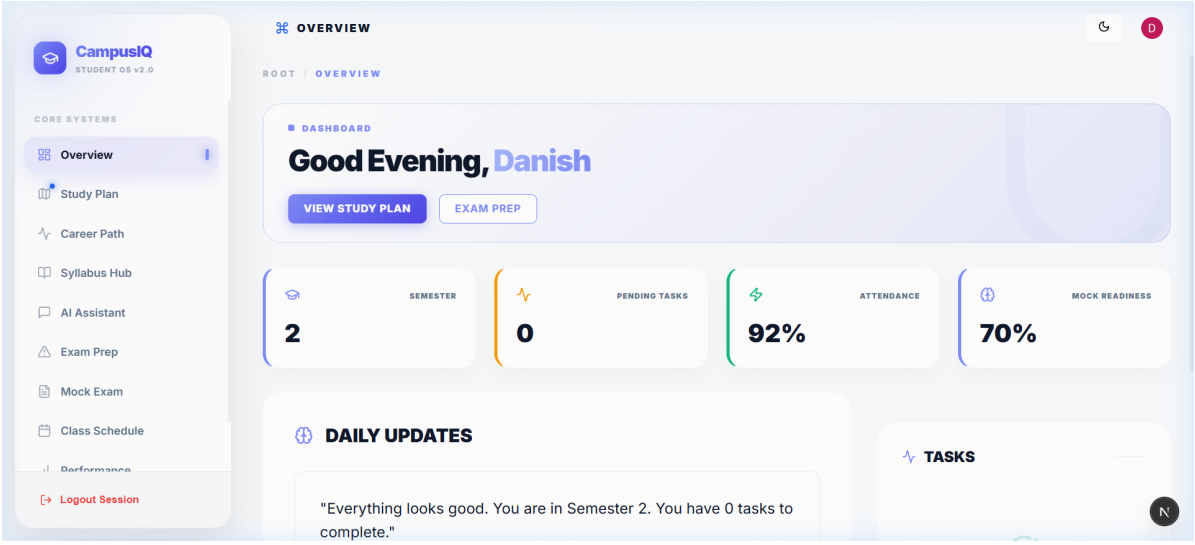

Figure 13.4: Dashboard cockpit overview

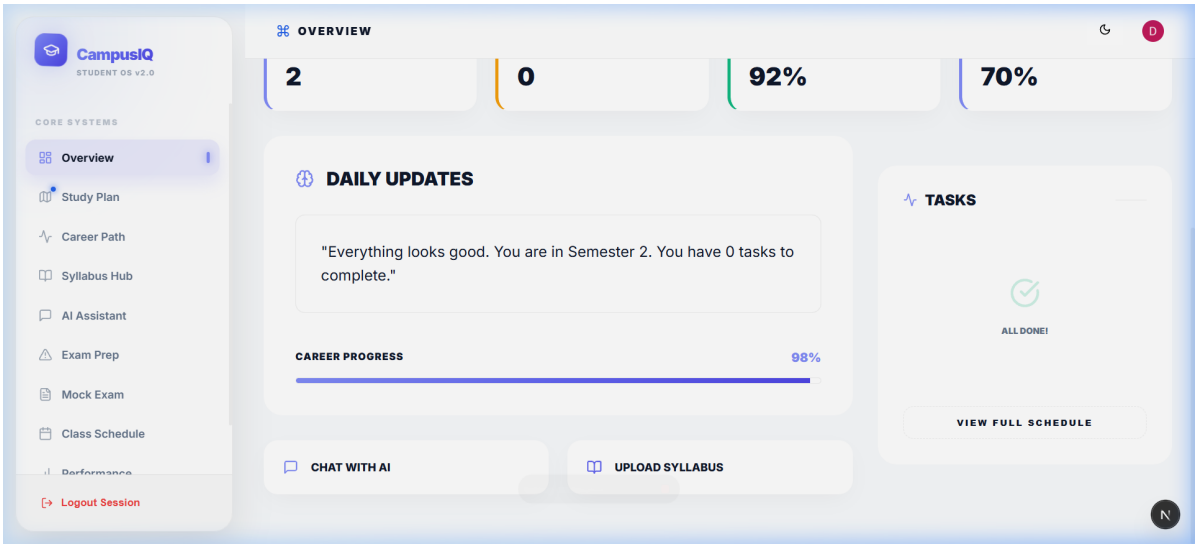

Figure 13.5: Dashboard telemetry stats bar

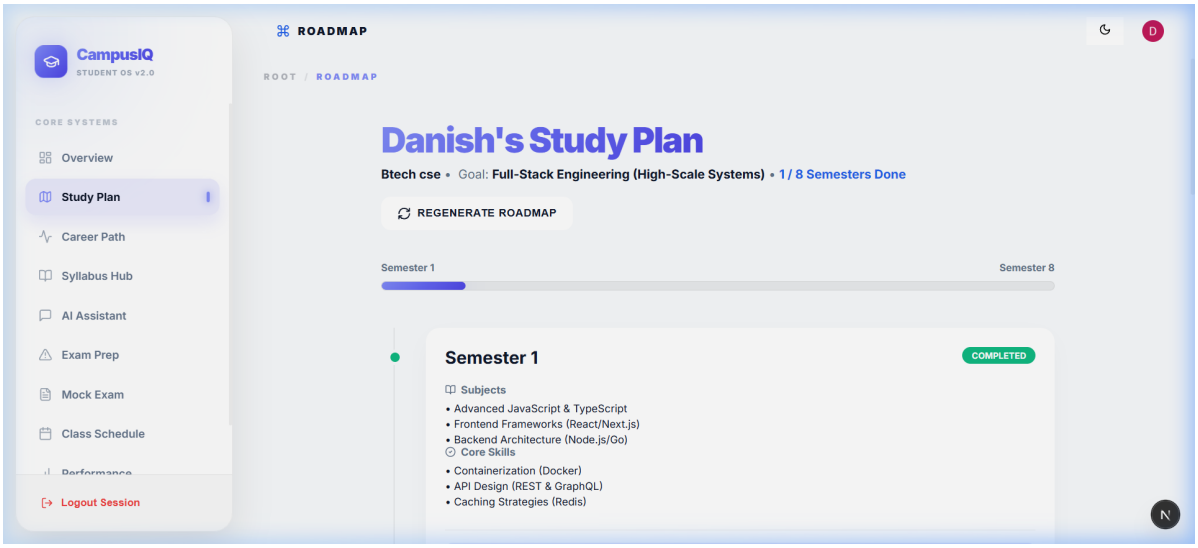

Figure 13.6: Roadmap overview

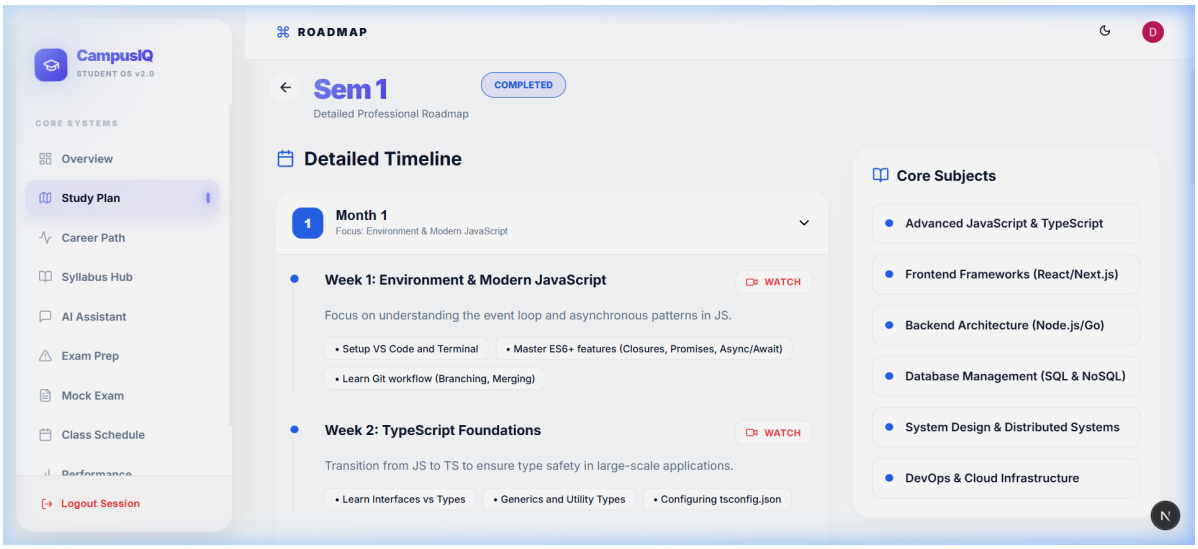

Figure 13.7: Roadmap detailed timeline

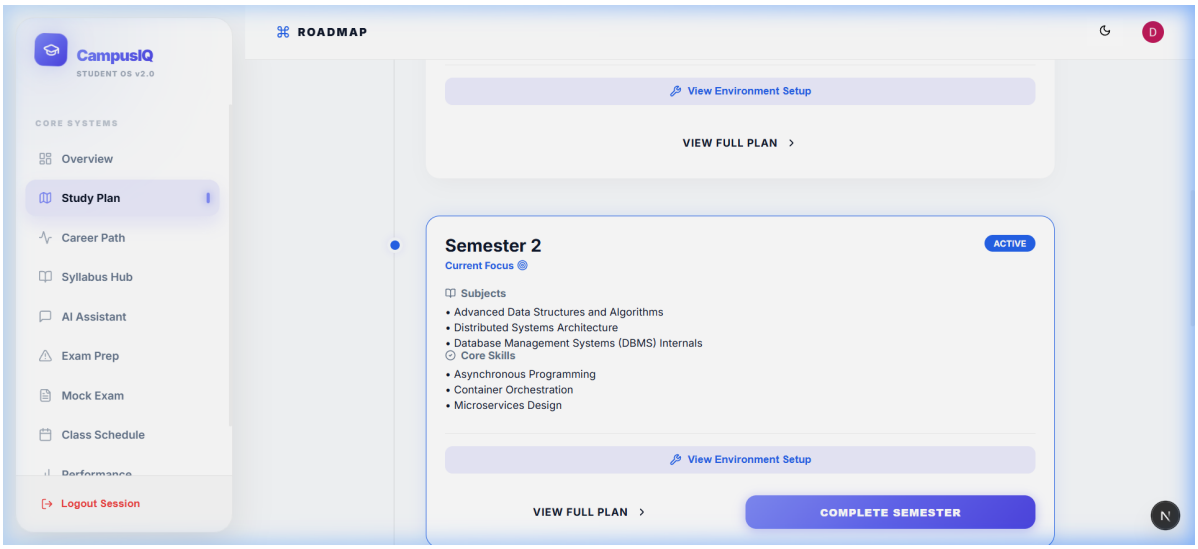

Figure 13.8: Active semester roadmap

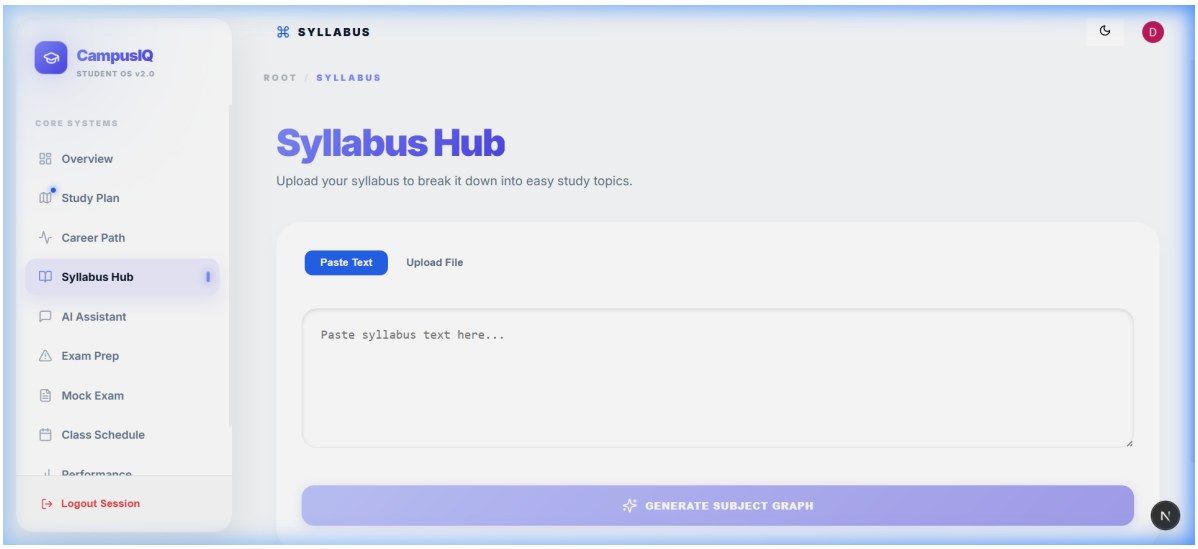

Figure 13.9: Syllabus intelligence hub

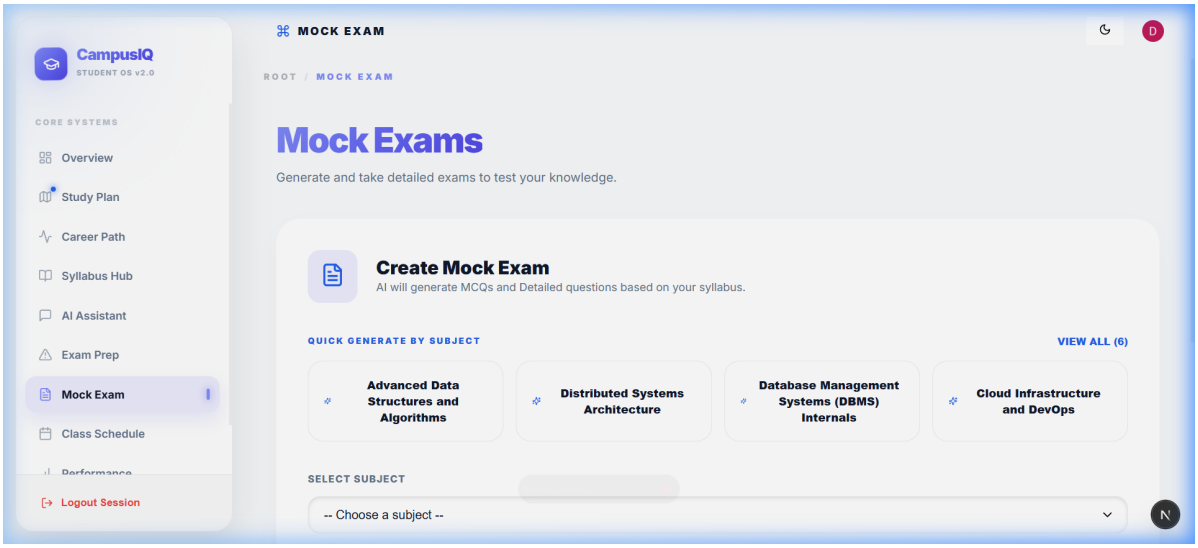

Figure 13.10: Mock exam hub

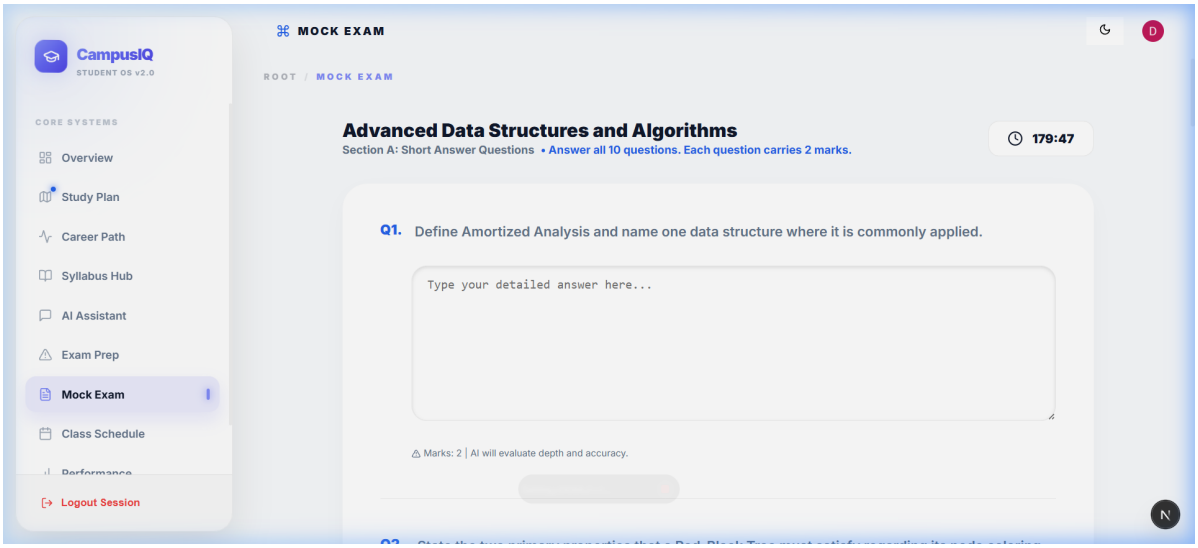

Figure 13.11: Mock exam question detail screen

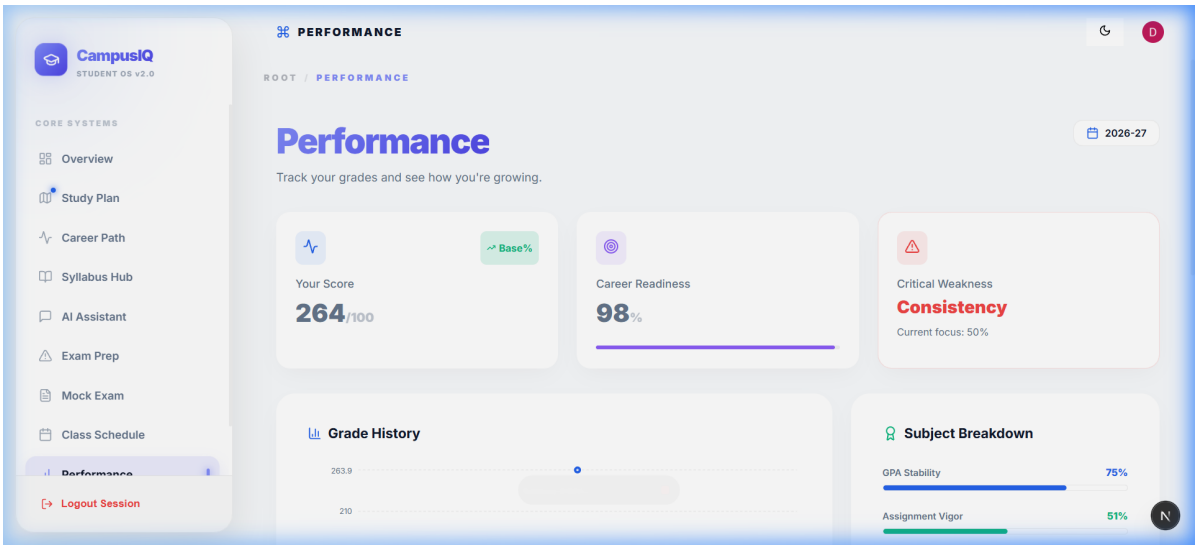

Figure 13.12: Analytics cockpit

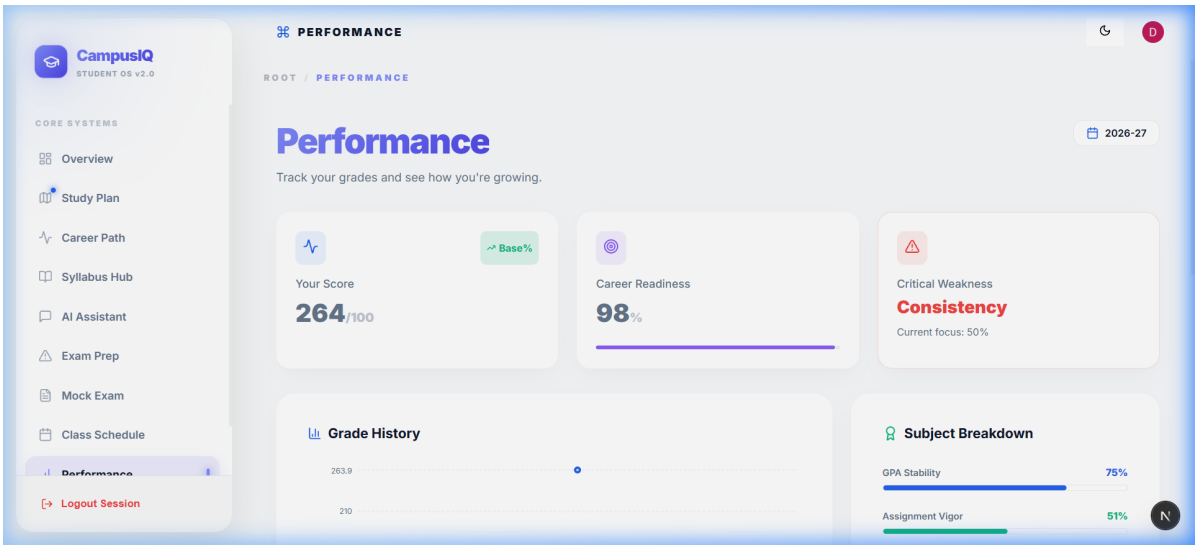

Figure 13.13: Subject breakdown chart

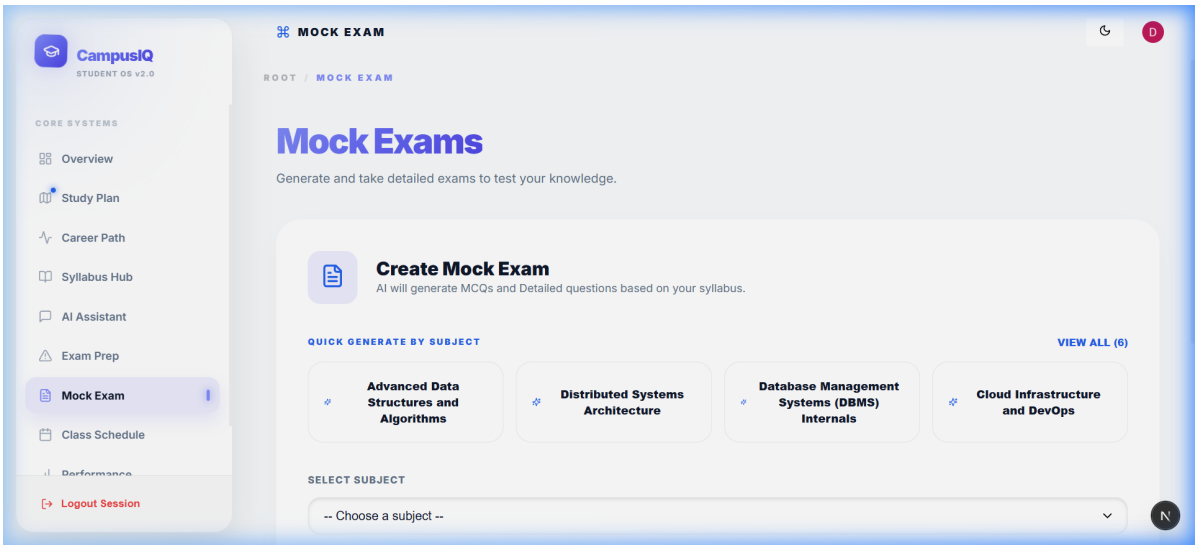

Figure 13.14: Quick generate grid

14. Conclusion

CampusIQ successfully demonstrates the design and implementation of an AI-assisted academic operating system for students. The project integrates authentication, relational

data modeling, AI-based generation, dashboard workflows, syllabus analysis, exam simulation, subjective grading, timetable management, and analytics into a single coherent web application.

The major achievement of the system is not only that it stores academic data, but that it converts academic data into useful actions. A syllabus becomes structured topics. A career goal becomes a semester roadmap. A mock exam becomes feedback and analytics. This transformation shows how modern AI systems can support student decision-making when combined with reliable software engineering practices.

15. Limitation

• Advanced generation and grading depend on Gemini API availability and quota limits.

• AI-based subjective answer evaluation can vary and should be treated as guidance rather than final academic marking.

• The current system does not directly integrate with university ERP, attendance machines, or official grade portals.

• Most AI features require internet connectivity.

• Syllabus extraction quality depends on PDF clarity, formatting, and text availability.

• AI-generated roadmaps, references, and topic explanations should be reviewed before high-stakes use.

16. Future Scope

• University ERP integration for official attendance, grades, timetables, and examination data.

• Placement intelligence that maps semester subjects and projects to job descriptions and internship roles.

• Collaborative learning features such as group roadmaps and shared mock exam sessions.

• Native Android and iOS applications for reminders, offline notes, and quick assistant access.

• Advanced analytics for predictive risk detection in weak subjects, attendance shortage, and exam readiness.

• Faculty mode for approved syllabus templates, question banks, and evaluation rubrics.

• Retrieval-augmented learning using uploaded notes, textbooks, and previous papers.

17. Bibliography

Vercel, "Next.js Documentation," 2026. [Online]. Available: https://nextjs.org/docs.

Meta Open Source, "React Documentation," 2026. [Online]. Available: https://react.dev/.

Prisma Data, "Prisma ORM Documentation," 2026. [Online]. Available: https://www.prisma.io/docs.

Clerk, "Clerk Authentication Documentation," 2026. [Online]. Available: https://clerk.com/docs.

Google, "Gemini API Documentation," 2026. [Online]. Available: https://ai.google.dev/.

IEEE Computer Society, "IEEE Recommended Practice for Software Requirements Specifications," IEEE Std 830-1998.

R. S. Pressman and B. R. Maxim, Software Engineering: A Practitioner's Approach, McGraw-Hill Education.

I. Sommerville, Software Engineering, Pearson Education.

S. Russell and P. Norvig, Artificial Intelligence: A Modern Approach, Pearson.

M. Fowler, UML Distilled: A Brief Guide to the Standard Object Modeling Language, Addison-Wesley.

18. Plagiarism Report

Placeholder: The official plagiarism report is to be obtained from the University Library through the respective supervisor, as specified in the dissertation format notice.

Similarity Index: _____

Verified by: _____

Date: _____